TELOMERE ($TLMR): Backend Build

*Build spec for **Telomere_Overview.md**.*

Chain: Base mainnet (8453). Contracts in Foundry/Solidity, non-upgradeable except `RunEscrow` (UUPS behind a 48h timelock until audit, then frozen). USDC `0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913` (native Circle). ERC-8004 Identity `0x8004A169FB4a3325136EB29fA0ceB6D2e539a432`, Reputation `0x8004BAa17C55a88189AE136b182e5fdA19dE9b63`, Validation `0x8004Cc8439f36fd5F9F049D9fF86523Df6dAAB58`. Base Account spend permissions via Coinbase's open-source SpendPermissionManager (address to confirm at build (U)).

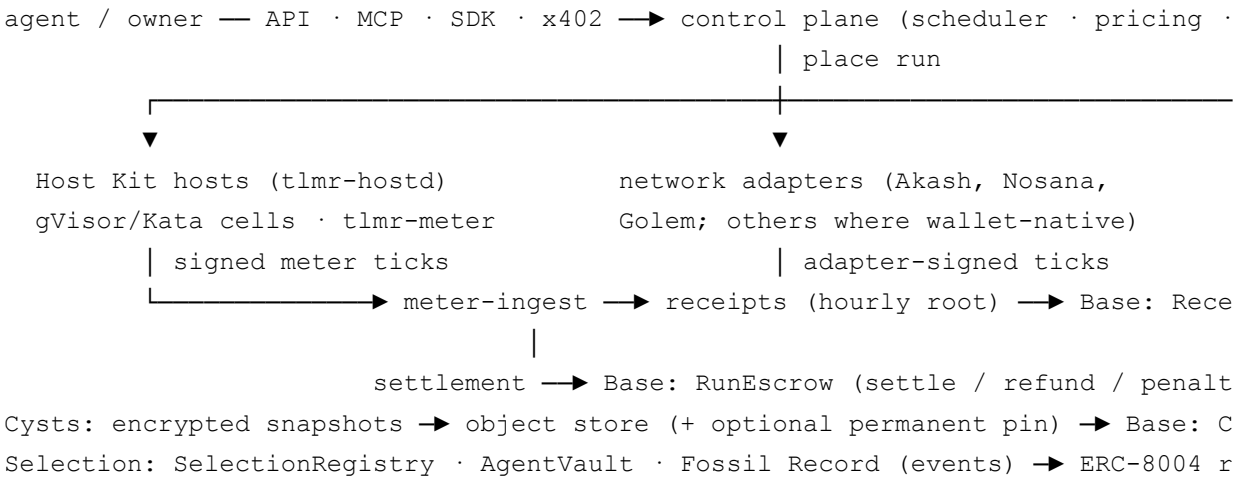
Off-chain: Bun · Hono · tRPC; Postgres + Timescale (meter series); Redis + BullMQ; Go for the Host Kit agent (`tlmr-hostd`); Rust for the in-cell meter (`tlmr-meter`); S3-compatible object storage for Cysts.

Payments: a **self-hosted x402 facilitator** (open-source x402 reference, extended), so no third-party account is needed. Schemes: `exact` (USDC EIP-3009) and `upto` (Permit2 witness, ceiling then actual). Solana USDC accepted for balance top-ups only (Solana Pay reference keys, finalized), credited on Base from treasury inventory.

(U) = unverified; confirm before mainnet.

---

0. Architecture



Laws:

1. **No settlement without a receipt.** Every billed second maps to a signed meter tick inside a receipt leaf.
2. **Refunds are default, not requested.** Unreceipted time is never charged.
3. **The leash is enforced in the contract**, not only in the API.
4. **Owners can always withdraw** unspent balances and kill an agent's runs; the platform's pause stops new runs only.
5. **Hosts never get user keys.** Cysts are encrypted to the agent's key; the host sees a running workload only on non-sealed cells.

---

1. Contracts

1.1 `HostRegistry`

```
struct Host {
  address operator; bytes32 hostKey;          // ed25519 key of tlmr-hostd (signs
  uint8   kind;                               // 0 HOST_KIT, 1 NETWORK_ADAPTER, 2
  bytes32 teeMeasurement;                     // sealed only: allowed image/measu
  uint96  collateral;                         // USDC, min 1,500 (GPU), 300 (CPU-
  uint32  strikes; uint64 unbondAt; bool active;
}
function register(bytes32 hostKey, uint8 kind, bytes32 teeMeasurement, uint96 col
function addCollateral(uint96 amt);
function requestUnbond();            // 21-day delay; blocked while runs are open
function withdrawCollateral();
function penalize(bytes32 hostId, uint96 amt, bytes32 evidenceLeaf, bytes32[] pro
```

1.2 `LeashRegistry`

```
struct Leash {
  address owner; address agent;              // agent = signer the API accepts fo
  uint96  maxPerHour; uint96 maxPerDay;      // USDC (6 dp)
  uint64  classMask;                         // allowed hardware classes (bitmask
  bool    sealedOnly; bytes32 imageAllowRoot; bytes32 regionAllowRoot;
  uint64  expiry; bool killed;
}
function setLeash(Leash calldata l);                // owner
function kill(bytes32 leashId);                     // owner, instant
function spentToday(bytes32 leashId) view returns (uint96);
```

`RunEscrow.open` and `RunEscrow.settle` both check the leash. A killed leash can't open

runs; open runs are stopped by the control plane within one meter tick and settled for receipted time only.

1.3 `RunEscrow`

```
function openFromPermit(RunTerms t, Permit2Witness sig);       // x402 upto: ceil
function openFromBalance(RunTerms t);                          // pre-funded Agen
function openFromSpendPermission(RunTerms t, SpendPermission p);// Base Account a
function settle(bytes32 runId, uint64 seconds, uint96 amount, bytes32 receiptLeaf
function refund(bytes32 runId);                                // permissionless
function settleBatch(Settlement[] calldata s);                 // long runs: vouc

struct RunTerms { bytes32 leashId; bytes32 hostId; uint16 hwClass; uint96 pricePe
                  uint96 ceiling; uint64 maxSeconds; uint64 openedAt; bytes32 ima
```

- `settle` pays `min(amount, ceiling)` where `amount = seconds × pricePerSec` and `seconds` must equal the sum of receipted billable seconds in the leaf; the remainder of the ceiling returns to the payer **in the same transaction**.
- If no settle arrives by `openedAt + maxSeconds + 6h`, anyone can call `refund` (full ceiling back).
- Platform fee is taken from the settled amount and sent to `FeeRouter`; the host share accrues to a pull-payment ledger (weekly claim).

1.4 `ReceiptRoot`

`postRoot(uint64 hour, bytes32 root, uint32 count)` by the receipts signer (rotated weekly; key in `SignerSet`). `verify(leaf, proof, hour)` view.

1.5 `CystRegistry`

`commit(bytes32 agentId, bytes32 cystHash, uint64 sizeBytes, bytes32 storageRef)` (signed by the agent key), `latest(agentId)`, `retire(agentId)`. Only hashes and pointers on-chain; contents encrypted off-chain.

1.6 `AgentVault` and `SelectionRegistry`

```
// One AgentVault per Selection agent (minimal clone). Receives income, pays RunE
function deposit(uint96 amt); function ownerWithdraw(uint96 amt);   // owner can
function payRun(RunTerms t) onlyAgentOrController;

// SelectionRegistry
function enter(uint256 erc8004Id, address vault, bytes32 leashId) payable; // ent
function spawn(bytes32 parentId, uint256 childErc8004Id, uint96 endowment, bytes3
```

```
function declareDormant(bytes32 agentId);   // permissionless: vault < floor and
function declareDeath(bytes32 agentId);     // permissionless: dormant for 30 day
function revive(bytes32 agentId);           // owner top-up during dormancy only
event Birth(bytes32 id, bytes32 parent, uint96 endowment);
event Income(bytes32 id, uint96 amt, bytes32 source); event Burn(bytes32 id, uint
event Dormant(bytes32 id); event Death(bytes32 id, uint64 lifespan, uint96 lifeti
```

- **Offspring:** max 3 living children per agent; endowment paid from the parent's vault; child inherits a copy of the parent's Cyst (fork), never the parent's leash (the owner sets a new one, defaulting to the parent's caps halved).
- **Income** counts only USDC arriving in the vault from addresses other than the owner, the parent, the children and Telomere itself (circular funding excluded; owner top-ups are logged separately as "subsidy").
- **Fossil Record** = the event log, indexed into a public dataset: lifespan, lifetime income, lifetime burn, offspring, cause of dormancy/death.
- **$TLMR grace:** staking $TLMR in the agent's name extends dormancy before death from 30 to up to 60 days (it never adds funds).

1.7 `FeeRouter` and `$TLMR`

- `FeeRouter` : 80% to the work pool (hosts' uptime pool by receipted uptime share, probe runners, receipt verifiers, Cyst storage operators), 10% treasury, 5% $TLMR stakers, 5% buy-and-burn; distributed weekly.
- `$TLMR` : fixed-supply ERC-20; `TlmrStaking` with 14-day unstake; host scheduling priority within a price band = f(stake, strikes); owner fee-discount tiers.

Ops: fresh deployer; Safe 3-of-5 → 48h timelock for parameters, host minimums, fee rates and class definitions; guardian 2-of-3 can only pause `open*` and `register` , never `settle` , `refund` , withdrawals or `ownerWithdraw` .

---

2. Hosts and supply

2.1 Host Kit (`tlmr-hostd`)

- One installer (Docker or systemd) on Linux with NVIDIA driver ≥ 550. Generates an ed25519 host key, registers via `HostRegistry` , then pulls work.
- **Isolation:** CPU cells in gVisor (runsc); GPU cells in gVisor with `nvproxy` or Kata Containers with GPU passthrough. One tenant per GPU. Images allowlisted by digest (studio base images + owner-allowed digests).
- **Egress:** every cell routes through an egress proxy enforcing the leash's network

allowlist (default: package registries, model hubs, the agent's declared API hosts; SMTP and social-media APIs blocked unless allowlisted).

- **Meter (`tlmr-meter` , runs outside the cell):** every 5s emits `Tick{runId, seq, ts, cpu_ms, gpu_util, gpu_mem, pwr_w, state}` signed by the host key; the control plane counter-signs on receipt. A second, in-cell heartbeat from the workload's init process confirms the cell is alive.
- **Billable second** = a second covered by two consecutive valid ticks with the cell in RUNNING state and the in-cell heartbeat present.

2.2 Probes (honesty checks, non-TEE hosts)

Probe	When	Pass condition
Class benchmark	At admission, then randomly ≥ 1/day	Fixed-seed GEMM + memory-bandwidth kernel inside the class band (e.g., H100 SXM FP16 dense within published range ±15%)
Per-run warm check	First 20s of each GPU run (billed to the platform, not the user)	Short kernel within band; otherwise the run moves to another host and the first host gets a strike
Memory reality check	Random	Allocate and verify the claimed VRAM
Deterministic re-execution	Runs flagged `deterministic` by the user (opt-in), 2% sample	Output digest matches a second host
Liveness	Continuous	Ticks every 5s; 3 missed ticks = host fault

Penalties (from collateral, via `penalize` with the evidence leaf): failed boot or host vanishes mid-run → 2× the run's settled-so-far amount to the user, capped at 25 USDC per event; benchmark below band → 5 USDC + strike; re-execution mismatch → 15% of collateral + suspension pending review. 3 strikes in 30 days → delisted.

2.3 Sealed hosts

- Intel TDX confidential VM + NVIDIA confidential computing (H100/H200/B200). Boot measures the Host Kit image into the TDX event log; the quote and the GPU attestation report are fetched per boot and their digest (`tee_quote_digest`) goes in every receipt of that boot.

- Workload images are measured on start; Cyst decryption keys are released to the cell only after the agent's SDK verifies the quote (attestation-gated key release).
- Limits stated publicly: no CPU↔GPU device binding on current hardware; physical attacks out of scope.

2.4 Network adapters

Network	Mode	Notes
Akash	Wallet-native deployments (SDL), paid from a treasury wallet in the network's token	Confidential compute available since Jul 2026 (attestation tooling immature (U))
Nosana	Wallet-native jobs (Solana), per-second pricing	GPU inference/containers
Golem	Wallet-native requestor	CPU/GPU tasks
Others (io.net, Lium, Phala)	Only if usable without identity checks for the operator (U per network)	Otherwise excluded (studio rule)

- Adapters wrap each lease with the same meter semantics: the adapter signs ticks from the network's lease status + an in-cell heartbeat; a lease that drops is a host fault (penalty paid from an **adapter reserve** funded by 2% of adapter revenue, since networks don't post Telomere collateral).
- Treasury keeps working inventory of each network's token; prices are quoted to users in USDC with a ≤ 3% inventory buffer.

---

3. Scheduler and pricing

- Hosts post `price_per_sec` per class and lane (standard / interruptible). The scheduler picks by: class match → leash filters (sealed-only, region, image) → **effective price** = posted price × (1 + strike factor) ÷ reliability (30-day completed-run ratio) → stake priority within ±5% price bands.
- **Interruptible lane:** host may pre-empt with 30s notice; the Cyst agent checkpoints; settlement covers completed seconds only; the job resumes on the next host.
- **Deadline jobs:** user gives `deadline`, `max_spend`, `est_gpu_hours` (or a calibration run); the scheduler plans across standard + interruptible capacity and re-plans on pre-

emption; if the deadline becomes infeasible within `max_spend`, it stops and refunds the remainder, then reports.

- **Public price board:** per class, per lane, median and p10/p90 per-second prices over 1h/24h/7d, from settled receipts only.

---

4. Cysts (persistence)

- **Contents:** the cell's writable filesystem layer (overlay diff), declared state directories, and optionally process state via CRIU (CPU) and NVIDIA `cuda-checkpoint` for CUDA processes (support varies by driver and workload (U)); otherwise apps checkpoint themselves via the SDK hook `onCyst()`.
- **Format:** zstd tarball → encrypted with XChaCha20-Poly1305 under a key wrapped to the agent's public key (and the owner's) → stored in object storage (2 regions) → optional permanent pin → `CystRegistry.commit`.
- **Triggers:** every N minutes (default 15), on interruptible pre-emption notice, on leash budget exhaustion, on Metabolism step-down, on owner request.
- **Resume:** pull, verify hash, decrypt inside the new cell (on sealed hosts only after quote verification), restore. Target < 20s for ≤ 2GB.
- **Rolling leases:** a "Lifeline" setting keeps an agent running indefinitely by renewing runs before the 24h mark on the same or another host, checkpointing across the hop.

4.1 Metabolism

Tier	Trigger (vault balance ÷ current burn per hour)	Allowed compute
Full	> 72h	Any class the leash allows
Lean	24–72h	One class down; GPU only for jobs flagged essential
Torpor	2–24h	CPU only; wakes on schedule or on incoming payment
Cyst	< 2h	Frozen; storage only; wakes on top-up

The **telomere** shown to owners and in Selection = `vault_balance ÷ trailing-24h burn rate`, in hours.

---

5. Receipts

```
RunReceiptLeaf {
  run_id, host_id, hw_class, lane, image_digest,
  started_at, stopped_at, billable_seconds, price_per_sec, amount_usdc,
  probe_result (pass | fail | n/a), perf_score,
  tee_quote_digest (sealed only), output_digest (if declared),
  tick_root,             // Merkle root of host-signed ticks for this run
  stop_reason (done | user | leash | budget | preempt | host_fault | probe_fail)
}
```

- `receipts` service builds leaves, an hourly Merkle root over all leaves, and posts it to `ReceiptRoot`.
- Public verifier (CLI + web): checks tick signatures against `HostRegistry.hostKey`, the leaf against the root, and the settled amount against `RunEscrow` events.
- Owners can export receipts as CSV/JSON; Selection agents' receipts are public by default.

---

6. Payments

Path	Flow
x402 `upto`	`POST /v1/cells` → 402 with `{scheme:"upto", max, asset: USDC, payTo: RunEscrow}` → client signs Permit2 witness for `max` → facilitator `verify` → `RunEscrow.openFromPermit` → run → `settle` charges actual seconds, refunds the rest
x402 `exact`	Small fixed jobs (e.g., a 60-second CPU task): EIP-3009 `transferWithAuthorization` into escrow with a fixed ceiling; same settlement
Balance	Owner or agent vault pre-funds; runs open with `openFromBalance`; long runs settle in batches every 15 min
Spend permission	Owner grants Base Account allowance (period, allowance) with `RunEscrow` as spender; each run pulls only its ceiling; leash caps still apply
Solana top-up	Solana Pay transfer with a reference key → credited to the Base balance from treasury inventory after finalization

Facilitator endpoints: `/supported`, `/verify`, `/settle`; non-custodial (funds go payer →

escrow, never through the facilitator's own wallet).

---

7. Data model (Postgres)

```
hosts(id, operator, host_key, kind, classes, region, collateral_usdc, strikes, re
host_prices(host_id, hw_class, lane, price_per_sec, updated_at)
leashes(id, owner, agent, max_per_hour, max_per_day, class_mask, sealed_only, ima
runs(id, leash_id, host_id, hw_class, lane, image_digest, ceiling, price_per_sec,
     billable_seconds, amount, stop_reason, escrow_tx, settle_tx, refund_tx)
ticks(run_id, seq, ts, cpu_ms, gpu_util, gpu_mem, pwr_w, state, host_sig, cp_sig)
probes(id, host_id, kind, ts, result, score, evidence_uri)
penalties(id, host_id, run_id, amount, reason, evidence_leaf, tx)
cysts(id, agent_id, hash, size_bytes, storage_ref, created_at, trigger)
receipts(leaf_hash, run_id, hour, root, proof)
selection_agents(id, erc8004_id, vault, owner, parent_id, tier, telomere_hours, s
selection_events(id, agent_id, kind, amount, ref, ts)    -- birth/income/burn/dor
```

One writer per table (scheduler owns `runs` state transitions; settlement owns tx columns; meter-ingest owns `ticks`).

---

8. API, SDK and MCP

REST (`/v1`):

- `POST /cells {class, lane, image, max_usd, max_seconds, leash_id, deterministic?, sealed_only?, cyst_from?}` → 402 or `{run_id, host, ssh|exec_url, price_per_sec}`
- `POST /cells/{id}/exec` (stream), `POST /cells/{id}/stop`, `GET /cells/{id}` (live seconds, spend, telomere)
- `POST /jobs/deadline {image, cmd, deadline, max_usd, est_gpu_hours?}`
- `POST /cysts/{agent}/snapshot`, `POST /cysts/{agent}/resume`
- `POST /leashes`, `POST /leashes/{id}/kill`
- `GET /receipts/{run_id}` (+ proof), `GET /prices`, `GET /hosts/{id}`
- `POST /selection/enter`, `POST /selection/{id}/spawn`, `GET /selection/fossils`

SDKs (TS/Python): `cell = await telomere.cell({class:"H100", maxUsd: 5})`, `cell.exec()`, `cell.cyst()`, `telomere.onCyst(handler)`, Leash helpers, receipt verifier.

MCP server (Streamable HTTP, x402 or key): `telomere_quote`, `telomere_run`,

`telomere_exec`, `telomere_stop`, `telomere_status`, `telomere_cyst_save/resume`, `telomere_receipt`, `telomere_leash_get`, `telomere_prices`. Every tool response includes `seconds_billed`, `spend_usd`, `telomere_hours`.

Agent-framework adapters: starter templates for ElizaOS, the open-source automaton framework and LangGraph agents: "run my own compute from my own vault".

---

9. Selection operations

- **Entry:** owner registers the agent's ERC-8004 identity, deploys an `AgentVault`, sets a leash (Selection maximums: $20/hour, $200/day), pays the entry fee, and declares income endpoints (x402 services, marketplace profiles).
- **Policy enforcement:** egress allowlist mandatory; outbound email/DM/social posting blocked unless the owner verifies the destination account belongs to them; automated content scan on outbound HTTP bodies for prohibited content; rate limits per destination.
- **Freeze:** guardian can freeze a Selection agent for a policy breach (publishes the reason and evidence hash); a frozen agent's vault remains withdrawable by its owner.
- **Public data:** live population, births/deaths per day, median lifespan, income vs burn distribution, top earners by verified income; all derived from on-chain events and receipts. Weekly CSV + a Dune-style public dataset.
- **Reputation:** each death writes a summary to ERC-8004 Reputation for that agent (lifespan, income, burn, cause); never self-reported numbers.

---

10. Services

Service	Job
`api` / `mcp`	Public surfaces; x402 challenge issuance
`facilitator`	x402 verify/settle (`exact`, `upto`)
`scheduler`	Placement, re-planning, deadline jobs
`leash-guard`	Mirrors on-chain leashes; stops runs on kill within one tick
`meter-ingest`	Validates and counter-signs ticks; detects faults
`probes`	Admission and random probes; re-execution sampling

`settlement`	Builds `settle` / `settleBatch` / `refund` / `penalize` transactions
`receipts`	Leaves, hourly roots, public verifier backend
`cyst-store`	Snapshot ingest, replication, lifecycle
`adapters-*`	Akash, Nosana, Golem (+ others only if identity-free)
`treasury`	Network-token inventory, Solana top-up crediting, weekly host payouts
`selection-indexer`	Fossil Record dataset, population stats, ERC-8004 summaries

---

11. Caps until audit

Item	Cap
Single run ceiling	250 USDC
Per leash per day	1,000 USDC
Agent vault (Selection)	2,500 USDC
Total escrowed at once	500,000 USDC
Host collateral considered for scheduling	25,000 USDC

12. Tests and launch gates

- **Escrow:** settle can't exceed ceiling or receipted seconds; refund always available after timeout; a killed leash can't open; property-based fuzzing of `settle` / `refund` races.
- **Meter:** fault injection (host vanishes, network partition, clock skew, replayed ticks) never bills unreceipted seconds.
- **Probes:** a throttled GPU (power-capped to 50%) fails the warm check; a host claiming more VRAM fails the memory check.
- **Cysts:** resume after kill -9 on a different host; corrupted snapshot rejected by hash; sealed resume refuses an unverified quote.
- **Selection:** circular-funding fixture is excluded from income; death and revival

transitions; offspring cap; owner withdrawal works while frozen.

- **Gates:** two audits (escrow + registries), verified contract sources, public receipt verifier, refund-rate dashboard live.

13. Build order (Claude-Code-sized milestones)

#	Milestone	Lane
M1	Contracts: HostRegistry, LeashRegistry, RunEscrow, ReceiptRoot, FeeRouter + Foundry tests	Chain
M2	Host Kit (gVisor CPU cells) + meter + meter-ingest	Host
M3	Facilitator (`exact` , `upto`) + API + settlement + receipts → **CPU launch**	Core
M4	GPU cells (gVisor nvproxy / Kata) + probes	Host
M5	Cysts + Lifeline + Metabolism	Core
M6	Network adapters (Akash, Nosana, Golem) + treasury	Supply
M7	Sealed hosts (TDX + NVIDIA CC) + attestation-gated key release	Host
M8	Interruptible lane + deadline jobs + price board	Core
M9	AgentVault + SelectionRegistry + selection-indexer + Fossil Record	Selection
M10	MCP + SDKs + framework templates; $TLMR + staking	Distribution